

ADR-001: Modularização da Arquitetura do SGDI e Desacoplamento em Camadas de Serviço

Data

07/10/2026

Status

Aceito

Contexto

Durante o ciclo de desenvolvimento da Sprint 4, a aplicação SGDI (Sistema de Gestão de Demandas Internas) incorporou funcionalidades críticas de governança, analytics e exportação corporativa:

- Painel analítico executivo com 8 cartões de telemetria em tempo real;
- 5 gráficos dinâmicos (Chart.js) com suporte a séries históricas e reatividade assíncrona;
- Regras de negócio de SLA por prioridade (Crítica: 2d, Alta: 5d, Média: 10d, Baixa: 20d);
- Ponderação de severidade do backlog ativo (pesos 4x, 3x, 2x, 1x) e regra de exclusão de canceladas do tempo médio;
- Regra de violação imediata de SLA para demandas sem responsável atribuído;
- Visão dedicada (War Room) de contenção para demandas críticas e/ou atrasadas com triagem e alocação inline;
- Motor multiformato de exportação corporativa com emissão instantânea de relatórios em PDF (`markdown_pdf`), planilhas formatadas em Excel (`openpyxl`) e arquivos estruturados em CSV (`utf-8-sig`).

Como resultado direto dessa expansão de funcionalidades, o arquivo principal `app.py` atingiu mais de 1.400 linhas de código, concentrando em uma única unidade de compilação:

1. Roteamento e controladores HTTP do framework Flask;
2. Conexão, queries SQL complexas, migrações e backfill de banco de dados SQLite;
3. Regras de negócio de SLA, pesos de criticidade e cálculo de prazos regimentais;
4. Agregações estatísticas, agrupamentos temporais e telemetria analítica;
5. Geração, estilização e manipulação binária de arquivos para exportação (PDF, Excel, CSV).

Essa sobrecarga monolítica causou graves violações arquiteturais:

- **Violação do Princípio da Responsabilidade Única (SRP - Single Responsibility Principle):** Qualquer alteração visual, ajuste de query SQL, correção na fórmula de SLA ou formatação de planilha exigia editar o mesmo arquivo `app.py`.
- **Dificuldade de Testes Unitários e Isolados:** Não era viável testar a lógica de cálculo de tempo médio, fórmulas de severidade de SLA ou geradores de planilha sem carregar o contexto de rotas do Flask.

- **Risco de Conflitos de Merge:** Múltiplos desenvolvedores atuando em tarefas distintas (ex.: ajustes no layout versus melhorias no cálculo de SLA) geravam conflitos recorrentes no Git.
 - **Alto Custo de Manutenção Cognitiva:** A navegação em um arquivo de 1.400 linhas tornou-se lenta e suscetível a erros acidentais de escopo ou dependências cruzadas.
-

Decisão

Decidimos refatorar e desacoplar o monolítico `app.py`, adotando uma **Arquitetura em Camadas de Serviço (Layered Service Pattern)**, mantendo 100% de compatibilidade retroativa com as rotas, endpoints e templates existentes:

```
sistema_sgdi/
├── app.py                # Controlador HTTP fino (Flask): orques
├── config.py             # Centralização de constantes corporati
├── database.py           # Persistência de dados: conexões, migr
├── services/
│   ├── __init__.py
│   ├── sla_service.py    # Regras de SLA, cálculo de prazos, par
│   ├── analytics_service.py # Filtros dinâmicos, métricas gerais, s
│   └── export_service.py  # Motores de exportação corporativa: CS
```

Divisão das Camadas e Responsabilidades:

1. Camada de Configuração (`config.py`):

- Centraliza constantes de ambiente, prazos de SLA por prioridade, pesos numéricos de criticidade (`PESO_PRIORIDADE`), listas de opções de status/prioridade e metadados institucionais da empresa.
- Elimina valores "mágicos" (hardcoded) espalhados pelo código.

2. Camada de Acesso a Dados (`database.py`):

- Encapsula o acesso ao SQLite (`get_db`), executa rotinas de migração e backfill (`garantir_migracao_db`) e padroniza a query principal de demandas (`SELECT_DEMANDAS`) com cálculos condicionais de SLA em SQL.

3. Camada de Regras de Negócio e SLA (`services/sla_service.py`):

- Fornece funções puras e isoladas para tratamento e conversão resiliente de datas (`parse_data`), formatação amigável de tempos em linguagem natural (`formatar_tempo_resolucao`) e determinação de prazos regimentais (`calcular_prazo`).

4. Camada de Inteligência Analítica (`services/analytics_service.py`):

- Responsável por montar filtros SQL dinâmicos (`construir_filtro_sql`) e consolidar o payload estatístico de telemetria (`obter_metricas_analytics`), abrangendo contadores de ciclo de vida, agrupamento por responsável, métricas de SLA e evolução temporal para o Chart.js.

5. Camada de Exportação Corporativa (`services/export_service.py`):

- Isola os geradores de documentos binários (`gerar_csv_response`, `gerar_excel_response`, `gerar_pdf_response`), aplicando formatação de células, cabeçalhos institucionais, notas de rodapé com filtros ativos e timestamps.

6. Camada de Apresentação e Controle (`app.py`):

- Reduzido de 1.407 para aproximadamente 370 linhas.
 - Atua exclusivamente como controlador web, recebendo parâmetros via requisições HTTP, delegando a execução para a camada de serviços correspondente e entregando respostas (templates HTML ou streams binários).
-

Alternativas Consideradas

Opção 1: Manter o Monólito `app.py` Apenas com Comentários de Seção

- **Prós:**

- ☐ Nenhum arquivo adicional no projeto.
- ☐ Dispensa o gerenciamento de novos módulos de importação.

- **Contras:**

- ☐ Mantém o alto acoplamento e a violação contínua do Single Responsibility Principle.
- ☐ Impede a criação de testes unitários isolados para regras de negócio sem subir o servidor web.
- ☐ Dificulta a evolução do sistema para novas sprints e colaboração entre múltiplos membros.

- **Por que não:** É incompatível com as boas práticas de engenharia de software e padrões corporativos exigidos em auditorias técnicas e desafios de inovação.

Opção 2: Microserviços ou Múltiplos Blueprints Isolados em Pacotes Complexos

- **Prós:**

- ☐ Isolamento extremo entre módulos com ciclo de vida independente.

- **Contras:**

- ☐ Complexidade arquitetural desproporcional para o tamanho atual da aplicação (overengineering).
- ☐ Risco de quebra de referências em chamadas `url_for()` amplamente utilizadas nos templates Jinja2 já desenvolvidos.
- ☐ Overhead desnecessário de deploy e configuração.

- **Por que não:** Introduziria complexidade acidental sem gerar benefícios proporcionais nesta fase do projeto.

Opção 3: Arquitetura em Camadas de Serviço (Layered Service Pattern)

com Controlador Conciso

- **Prós:**

- Separação limpa de preocupações (*Separation of Concerns*).
- Redução drástica da complexidade ciclomática de `app.py` (de 1.407 para ~370 linhas).
- Testabilidade imediata das funções de SLA e analytics.
- Preservação de 100% dos nomes de endpoints e parâmetros, garantindo zero impacto nos templates Jinja2 existentes.

- **Contras:**

- Criação de novos arquivos no repositório (`config.py`, `database.py`, pacote `services/`).

- **Por que foi a escolhida:** Oferece o equilíbrio ideal entre simplicidade, robustez, manutenibilidade e total compatibilidade retroativa.
-

Consequências

Positivas:

1. **Alta Coesão e Baixo Acoplamento:** Cada módulo agora tem uma única razão para mudar. Alterações no layout ou em rotas não afetam a lógica de SLA; mudanças na biblioteca de PDF ou Excel afetam apenas o `services/export_service.py`.
2. **Facilidade de Manutenção e Leitura:** `app.py` tornou-se um arquivo enxuto, onde a navegação entre rotas e respostas é imediata.
3. **Testabilidade Isolada:** As funções de SLA e inteligência analítica podem ser testadas unitariamente via scripts ou suítes de teste sem necessidade de mockar requisições Flask.
4. **Isolamento de Dependências Pesadas:** As bibliotecas `openpyxl` e `markdown_pdf` ficam confinadas exclusivamente no serviço de exportação, mantendo o controlador limpo.
5. **Prontidão para Escalar:** O SGDI ganha uma estrutura profissional preparada para as próximas sprints (novos módulos, autenticação, integrações de API).

Negativas (Trade-offs):

1. **Gerenciamento de Módulos:** Requer atenção na manutenção dos imports internos entre os pacotes (`from services.sla_service import ...`, `from config import ...`).
2. **Multiplicidade de Arquivos:** Pequeno aumento na quantidade de arquivos na raiz do projeto, plenamente compensado pela clareza estrutural obtida.